FIT5201 - Machine Learning

Module 2: Linear Models for Regression

Part A:

Provide deeper understanding of Linear Regression Models

- **Squared error**
- **Basis functions**
- **Parameter fitting**
- **Regularisation**

Part A

Linear Regression – Squared Error

Regression problem

Given

- data $D = (x_1, t_1), \ldots, (x_N, t_N)$
- sampled according to joint distribution of RVs X and T
- input variable(s) X often assumed with $D_X = \mathbb{R}^d$
- real-valued (continuous) target variable $T \in \mathbb{R}$

Goal

- learn prediction function $y: D_X \to \mathbb{R}$
- such that want that $y(X) \approx T$ on average; more precisely that minimizes generalisation error $\mathbf{E}\left(l\left(T, y(X)\right)\right)$

…in *parametric* regression

- learn fixed-dimensional parameter vector $\boldsymbol{w} \in \mathbb{R}^p$
- that determines prediction together with inputs, $y: D_X \times \mathbb{R}^p \to \mathbb{R}$

Recall: Model Fitting

Goal: minimal generalisation error (prediction risk), i.e., expected value of loss function l (e.g., mean squared error)

$$\mathbf{E}\left(l\left(T, y(X, \boldsymbol{w})\right)\right)$$

Parameter learning: minimise training error (empirical risk), i.e., mean loss value on training data

$$E(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} l\left(t_n, y(\boldsymbol{x}_n, \boldsymbol{w})\right)$$

loss function measures misfit

Common choice for regression: squared error (loss)

$$l\left(t_n, y(\boldsymbol{x}_n, \boldsymbol{w})\right) = \frac{1}{2}\left(t_n - y(\boldsymbol{x}_n, \boldsymbol{w})\right)^2$$

Why Squared Error?

Approach Lecture 1: minimise sum of squared errors

$$E(w) = \frac{1}{2}\sum_{n=1}^{N}(y(x_n, w) - t_n)^2$$

Approach Lecture 2: maximise (log) likelihood

$$L(w) = \sum_{n=1}^{N} \ln p(t_n \mid x_n, w)$$

Note:
we sometimes use sum and sometimes mean squared error

Can use approach 2 to justify approach 1

The expected squared error

For continuous X we can write generalisation error, i.e., expected squared error, as follows

$$\mathbf{E}\left(\left(T - y(X)\right)^2\right) = \int_{D_X}\int_{D_T}\left(t - y(x)\right)^2 p(x,t)\mathrm{d}t\mathrm{d}x$$

Why? The squared error $\left(T - y(X)\right)^2 = Z$ is a new random variable. Don't we have to find its density p_Z to compute

$$\mathbf{E}(Z) = \int_{D_Z} z\, p_Z(z)\mathrm{d}z$$

Expected values

Recall definition for numerical random variable X:

$$\mathbf{E}[X] = \int_{D_X} x\, p(x)\, dx \qquad\qquad \mathbf{E}[X] = \sum_{x \in D_x} x\, p(x)$$

continuous X

discrete X

Let f be some function. Then can compute expectation of $f(X)$ as follows:

$$\mathbf{E}[f(X)] = \int_{D_X} f(x)\, p(x)\, dx \qquad\qquad \mathbf{E}[f(X)] = \sum_{x \in D_x} f(x)\, p(x)$$

this is a new random variable

but can just use density or mass function of X to compute expectation

This is called Law of the Unconscious Statistician (LOTUS).

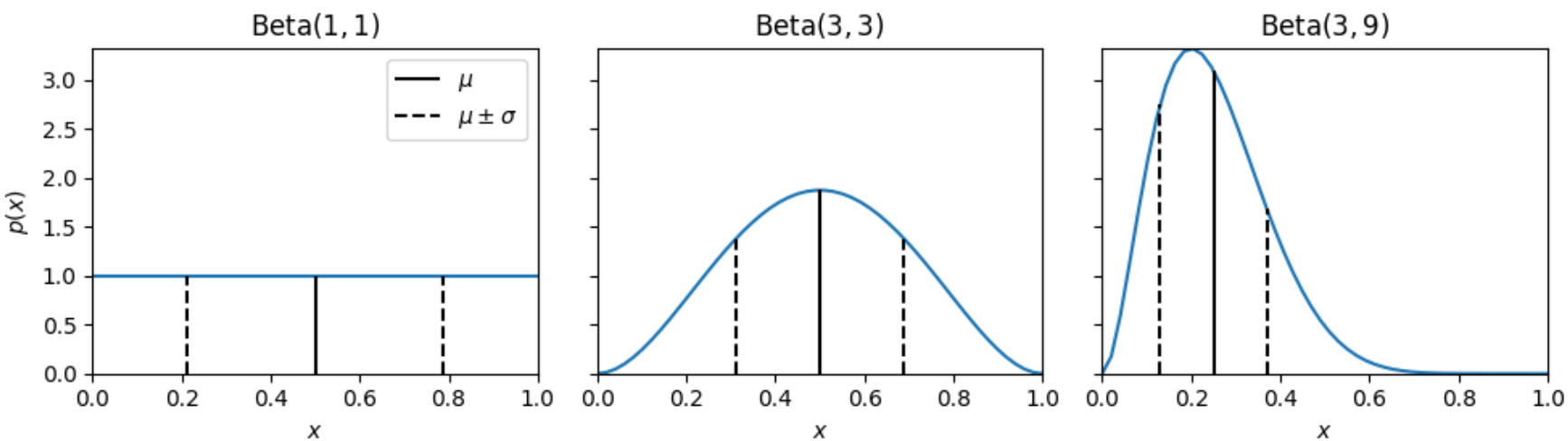
Variance and Standard Deviation

Variance and standard deviation of rv X measure "dispersion" of probability mass or density around its mean $\mu = \mathbf{E}[X]$:

$$\mathrm{Var}[X] = \mathbf{E}[(X-\mu)^2] \boxed{= \int_{D_x} (x-\mu)^2\, p(x)\mathrm{d}x \quad \text{or} \quad = \sum_{x \in D_x} (x-\mu)^2 p(x)}$$

$$\mathrm{Std}[X] = \sqrt{\mathrm{Var}[X]} = \sigma$$

expected squared error of the mean when predicting value for X



Probabilistic regression model

Assume target follows deterministic function plus random noise

$$T = y(X) + \varepsilon$$

What distribution should we assume for noise?

If we assume many small i.i.d. noise sources $\epsilon_1, \ldots, \epsilon_n$ with $\mu = \mathbf{E}[\epsilon_i] = 0$ and $\sigma_i = \mathrm{Var}[\epsilon_i] = \sigma^2/n$

$$\varepsilon = \epsilon_1 + \epsilon_2 + \cdots + \epsilon_n$$

Then, as $n \to \infty$, by "central limit theorem", the density of ε approaches

$$p(\epsilon) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$$

normal distribution
$\epsilon \sim N(0, \sigma^2)$

Squared Error and Maximum Likelihood

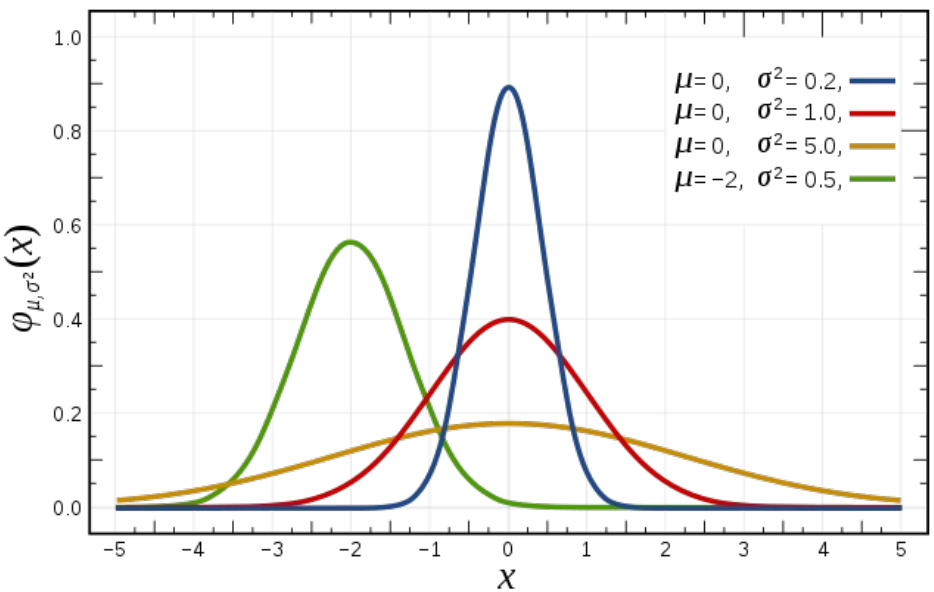
Assume target follows deterministic function plus random noise

$$T = y(X) + \varepsilon$$

where ε is normally distributed

$$\varepsilon \sim N(0, \sigma^2)$$

Gives rise to probabilistic model



$$p(t \mid x, w) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t - y(x, w))^2}{2\sigma^2}\right)$$

$$T \mid X = x \sim N(y(x, w), \sigma^2)$$

Squared Error and Maximum Likelihood

Probabilistic model

$$p(t \mid x, w) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t - y(x, w))^2}{2\sigma^2}\right)$$

Log likelihood (of individual data point)

$$\ln p(t \mid x, w) = \boxed{-\ln\sqrt{2\pi}\sigma - \frac{1}{\sigma^2}}\boxed{\frac{1}{2}(t - y(x, w))^2}$$

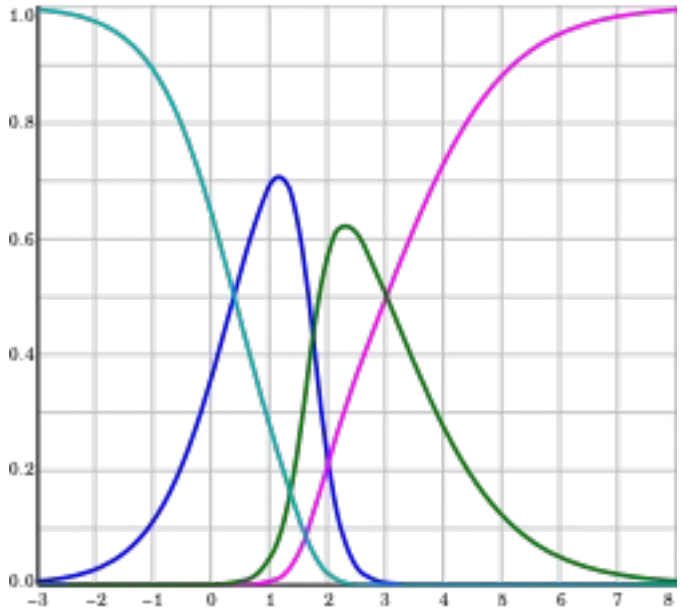
constants

squared error

$\Longrightarrow$ squared error minimisation is maximum likelihood estimation with additive normal (Gaussian) noise model

Part A

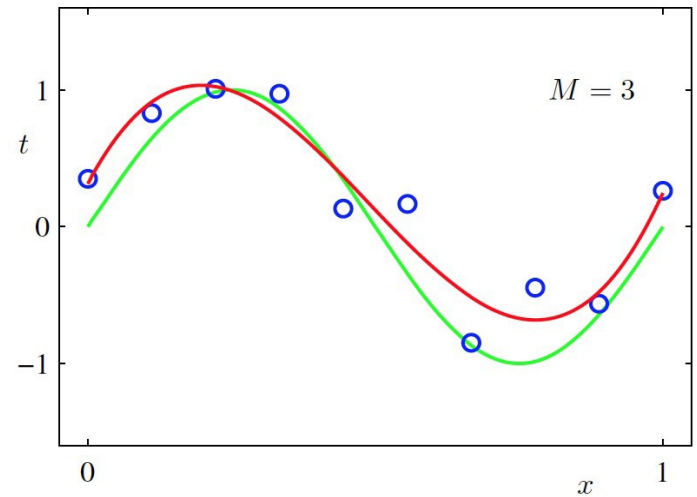
Linear Regression – Basis Functions



Linear Regression Model

Recall: to model $T = \sin 2\pi X + \varepsilon$ we used

$$y(x, \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M$$



Note:

- $y(x, \boldsymbol{w})$ <u>not</u> a linear function of inputs x
- $y(x, \boldsymbol{w})$ linear function of parameters $\boldsymbol{w}$

General linear regression model:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_1 \phi_1(\boldsymbol{x}) + w_2 \phi_2(\boldsymbol{x}) + \cdots + w_p \phi_p(\boldsymbol{x})$$

where $\phi_j(\boldsymbol{x})$ are called the basis functions. Can be rewritten as dot product

$$y(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{\phi}(\boldsymbol{x}) \cdot \boldsymbol{w} = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{w}$$

between weight vector $\boldsymbol{w} \in \mathbb{R}^p$ and feature vector $\boldsymbol{\phi}(\boldsymbol{x}) = \left[\phi_1(\boldsymbol{x}), \ldots, \phi_p(\boldsymbol{x})\right]^T$

Why this Form of Model?

Keeping model linear in parameters allows simple learning algorithm

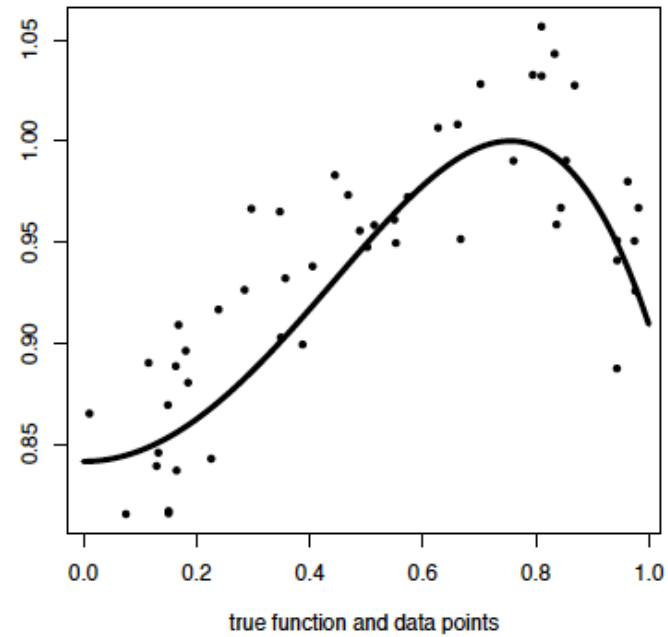
Introducing non-linear transformations of inputs allows to model complex relationships between X and T

Questions

- What basis functions to choose?
- How many?

Requires: Model Selection

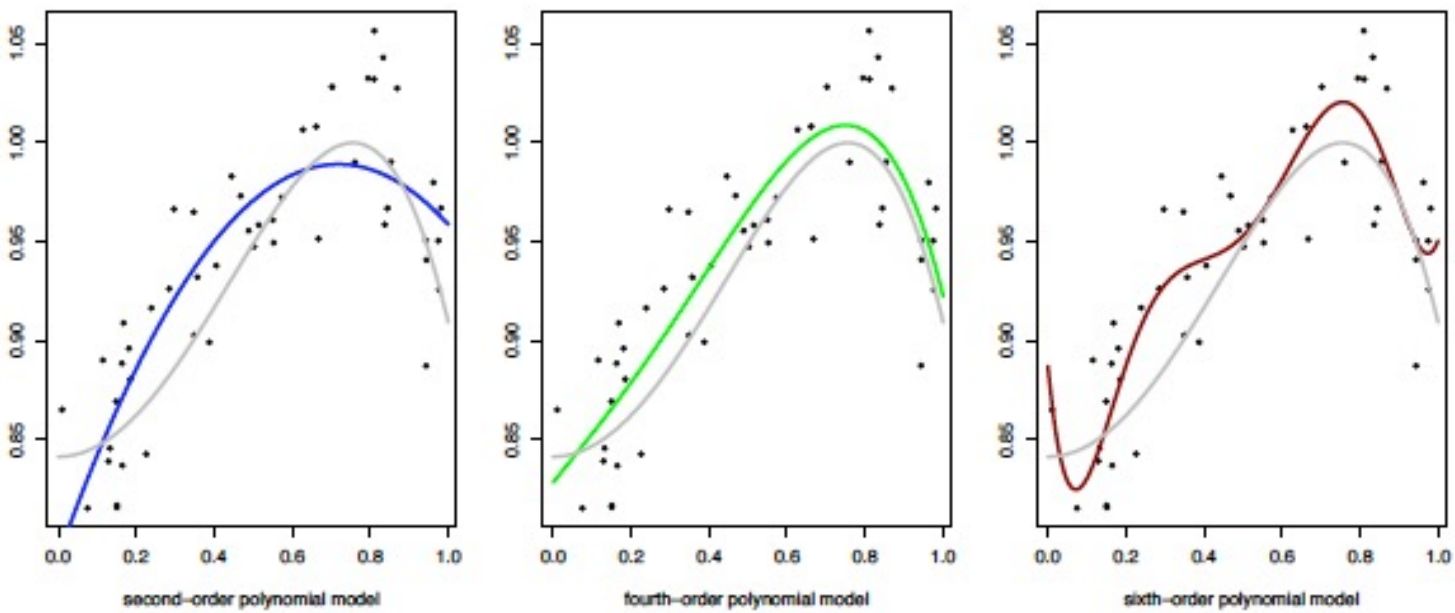
$$y = \sin(1 + x^2) + noise$$



Example Regression Models

- Regression curves for y having the form of

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1\phi_1(x) + w_2\phi_2(x) + \cdots + w_M\,\phi_M(x)$$



These polynomial models with increasing order can be viewed as basis function models with $\phi_j(x) = x^j$:

Popular Basis Functions

- Linear basis functions

$$\phi_j(x) = x_j$$

- Polynomial basis functions

$$\phi_j(x) = x^j$$ 1-dimensional inputs

$$\phi_j(x) = x_1^{k_1} x_2^{k_2} \dots x_d^{k_d}$$ general

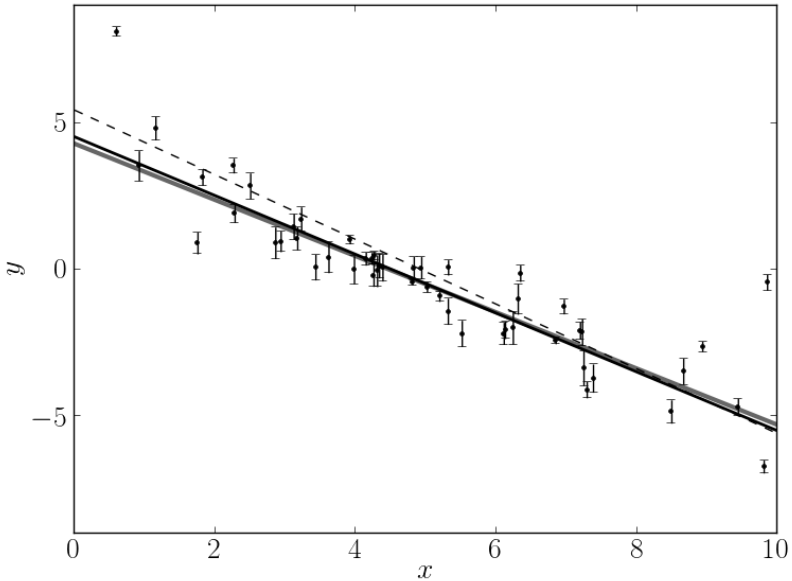
- Radial (Gaussian) basis functions

$$\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2s_j^2}\right)$$

- Others, e.g., sigmoidal or hyperbolic

Part A

Learning Model Parameters



Finding optimal weights

Derivative criterion

$$\nabla E(w) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \vdots \\ \frac{\partial E}{\partial w_p} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

Partial derivatives

$$\frac{\partial}{\partial w_i} E(w) = \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{n=1}^{N} \left(t_n - y(x_n, w)\right)^2$$

Finding optimal weights

Can rewrite partial derivative as dot or matrix product

residual

$$\frac{\partial}{\partial w_i} E(w) = -\sum_{n=1}^{N} (t_n - w \cdot \phi(x_n))\phi_i(x_n)$$
$$= -(\boldsymbol{t} - \boldsymbol{\Phi} w) \cdot \boldsymbol{\Phi}_i$$
$$= -\boldsymbol{\Phi}_i^T (\boldsymbol{t} - \boldsymbol{\Phi} w)$$

residual vector

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_p(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_p(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_p(x_N) \end{bmatrix} \qquad \boldsymbol{\Phi}_i = \begin{bmatrix} \phi_i(x_1) \\ \phi_i(x_2) \\ \vdots \\ \phi_i(x_N) \end{bmatrix} \qquad \boldsymbol{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix}$$

feature matrix feature column target vector

Finding optimal weights

Can rewrite partial derivative as dot or matrix product

$$\frac{\partial}{\partial w_i} E(w) = -\sum_{n=1}^{N} (t_n - w \cdot \phi(x_n))\phi_i(x_n)$$
$$= -\boldsymbol{\Phi}_i^T(\boldsymbol{t} - \boldsymbol{\Phi} w)$$

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_p(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_p(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_p(x_N) \end{bmatrix} \qquad \boldsymbol{\Phi}_i = \begin{bmatrix} \phi_i(x_1) \\ \phi_i(x_2) \\ \vdots \\ \phi_i(x_N) \end{bmatrix} \qquad \boldsymbol{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix}$$

Gradient criterion becomes

$$\nabla E(\boldsymbol{w}) = -\boldsymbol{\Phi}^T(\boldsymbol{t} - \boldsymbol{\Phi} w) = \boldsymbol{0}$$

normal equations

$$\boldsymbol{\Phi}^T \boldsymbol{\Phi} \boldsymbol{w} = \boldsymbol{\Phi}^T \boldsymbol{t}$$

system of linear equations
can be solved by
linear algebra software

$$\boldsymbol{w} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \boldsymbol{t}$$

Intermediate Summary

- Fitting criteria: find weights that minimise sum of squared errors
- Is equivalent to maximum likelihood assuming Gaussian noise
- How to identify minimum: derivative criterion
- For linear model with squared error: partial derivative i is dot product of feature column i with residual vector
- Derivative criterion: system of linear equations (normal equations)
- Can be solved by linear algebra algorithm (elimination or iterative)

Issues / Questions

- What if using different model or loss function?
- What if sample size is extremely large?

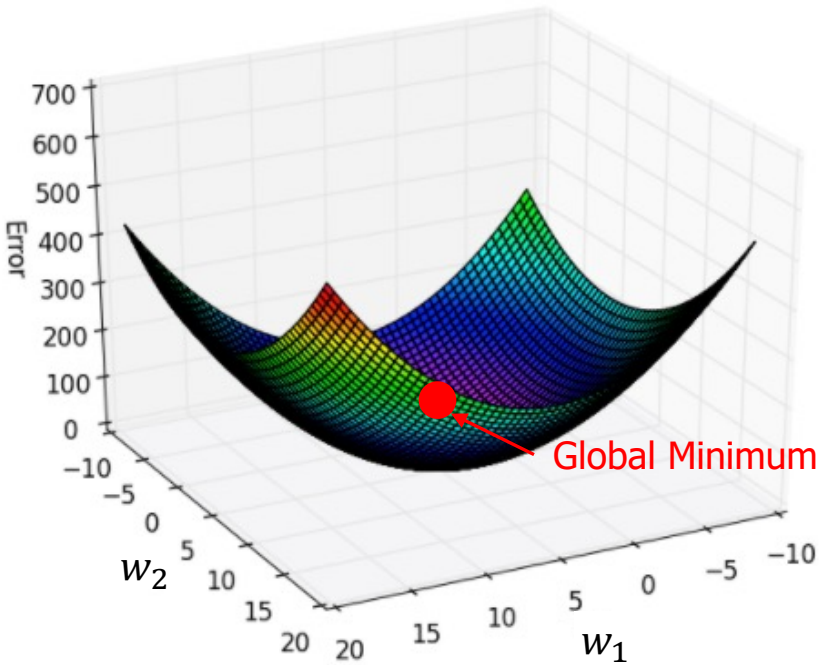
Can still use gradient-based approach

-> Gradient Descent / Stochastic Gradient Descent

A more general algorithm

For other models or loss functions: optimum not determined by linear system.

Gradient descent



How to compute downhill direction?

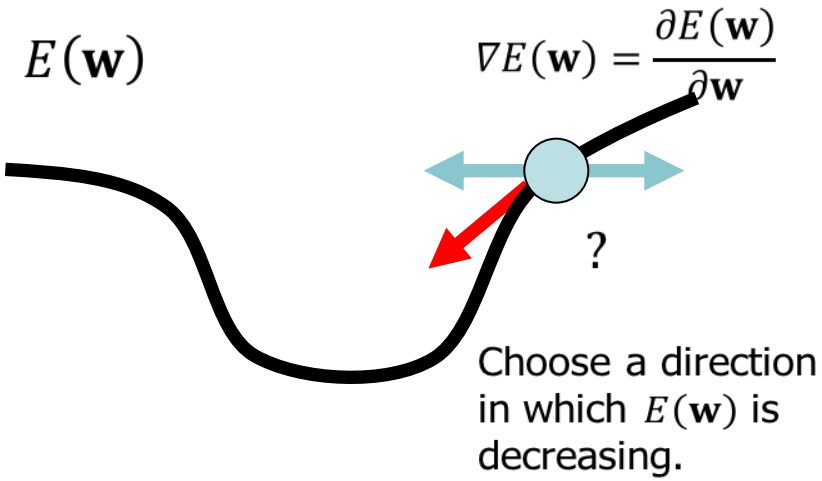
negative gradient of error at current point:

$$-\nabla E(w)$$

Idea: start from a random location on surface and move downhill to find point with lowest error.

Gradient Descent Algorithm

What does it mean to move downhill on the error function?
That is, how to change the parameter vector $\mathbf{w}$ to improve $E(\mathbf{w})$?



Gradient: direction of steepest ascent

(left figure is special case for one variable; where we only have two directions)

Gradient Descent Algorithm

Idea: start with some initial weights; then iteratively update using negative gradient of error as compass that always points downhill

Iterative **Algorithm**:

Eta is step-size or learning rate

```
choose initial weights w^(0)
for t = 1,2, ...
```

$$\boldsymbol{w}^{(t)} = \boldsymbol{w}^{(t-1)} - \eta \nabla E\left(\boldsymbol{w}^{(t-1)}\right)$$

```
    break when some condition is met
```

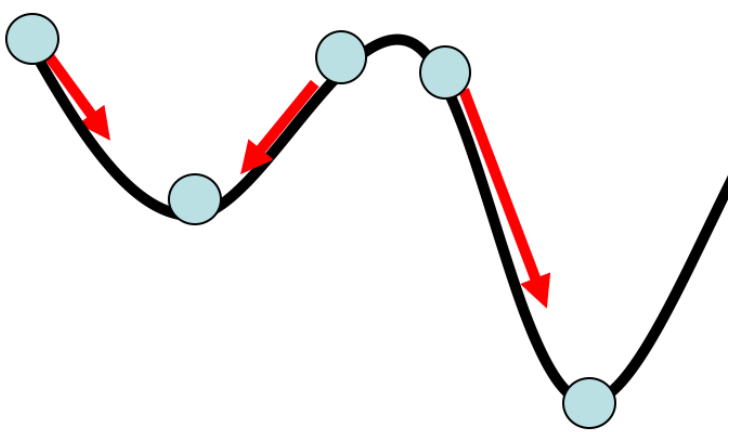
Questions:

1. How to initialize?
2. How to set learning rate?
3. When to stop?
4. How to compute gradient (for linear least squares not a problem)?

Gradient Descent Algorithm

How to choose initial weights?

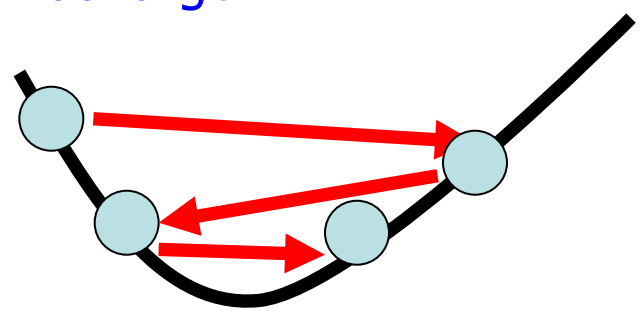
- sensitive decision: local minima
- not an issue for convex errors (like sum of squared errors)



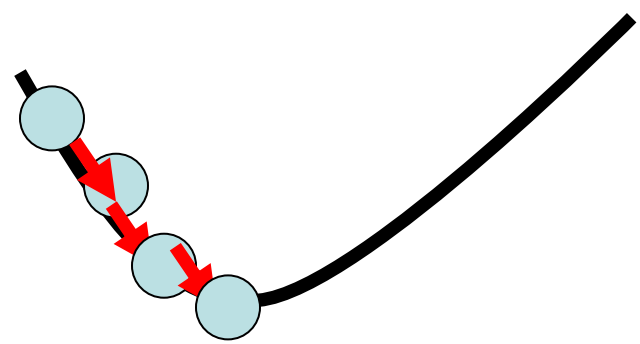
How to choose learning rate?

- static or change with time

Too large?

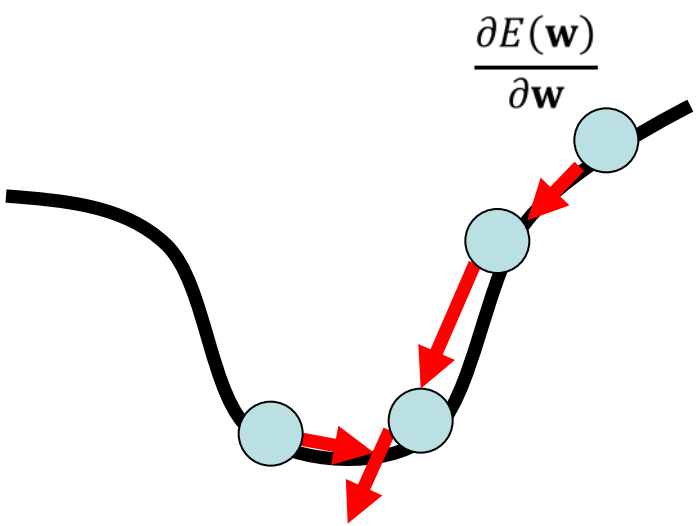


Too small?



Gradient Descent Algorithm

When to stop?




```
choose initial weights
```
$\boldsymbol{w}^{(0)}$

```
for
```
$t = 1,2, \ldots$

$$\boldsymbol{w}^{(t)} = \boldsymbol{w}^{(t-1)} - \eta \nabla E\left(\boldsymbol{w}^{(t-1)}\right)$$

```
break when
```
$\left|E\left(\boldsymbol{w}^{(t)}\right) - E\left(\boldsymbol{w}^{(t-1)}\right)\right| \leq \boldsymbol{\epsilon}$

Could also stop based on

- change in weights
- magnitude of gradient
- …

Intermediate Summary

Our motivation for iterative algorithm was

- ❑ More general algorithm (check)
- ❑ More efficient algorithm when N is very large

Problem

- ❑ Computing gradient still requires processing the entire training set
- ❑ Expensive and potentially wasteful for large N
- ❑ Maybe do not even have access to all data at the same time

$$\begin{aligned}\nabla E(\boldsymbol{w}) &= -\boldsymbol{\Phi}^T(\boldsymbol{t} - \boldsymbol{\Phi} w) \\ &= -\sum_{n=1}^{N} \left(t_n - \boldsymbol{w} \cdot \boldsymbol{\phi}(\boldsymbol{x_n})\right)\boldsymbol{\phi}(\boldsymbol{x_n})\end{aligned}$$

Idea

- ❑ Use approximate gradients (based on sample of training data)

Stochastic Gradient Descent (SGD)

Idea

- In each step, use approximate gradients
- …based on small random batch of data, say with indices $n_1, \dots, n_b$

$$
\begin{aligned}
\nabla E(\boldsymbol{w}) &= -\boldsymbol{\Phi}^T(\boldsymbol{t} - \boldsymbol{\Phi} w) \\
&= -\sum_{n=1}^{N} \left(t_n - \boldsymbol{w} \cdot \boldsymbol{\phi}(\boldsymbol{x_n})\right)\boldsymbol{\phi}(\boldsymbol{x_n}) \\
&\approx -\sum_{i=1}^{b} \left(t_{n_i} - \boldsymbol{w} \cdot \boldsymbol{\phi}\left(\boldsymbol{x_{n_i}}\right)\right) \boldsymbol{\phi}\left(\boldsymbol{x_{n_i}}\right) \\
&= -\widetilde{\boldsymbol{\Phi}}^T(\tilde{\boldsymbol{t}} - \widetilde{\boldsymbol{\Phi}} w) \\
&= \widetilde{\nabla} E(\boldsymbol{w})
\end{aligned}
$$

stochastic gradient

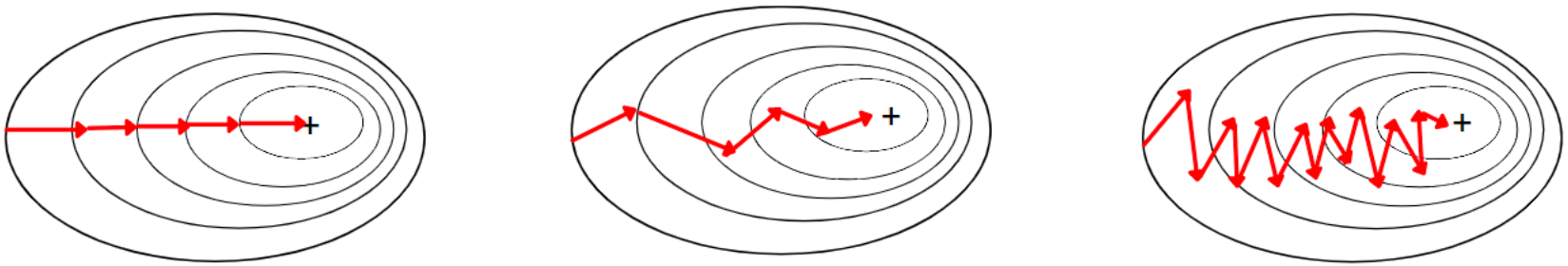
Observations

- Can choose batch size b arbitrary small and still have $\mathbf{E}[\widetilde{\nabla} E(\boldsymbol{w})] = \nabla E(\boldsymbol{w})$
- Can make individual updates very cheap. What is the downside?

SGD – Properties

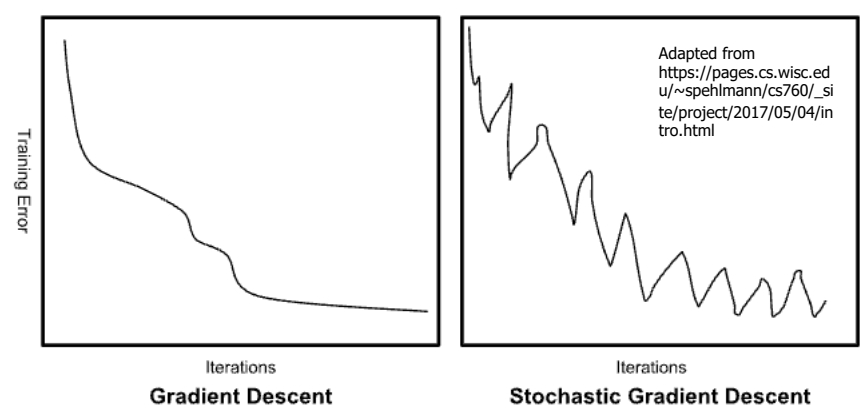
- Correct mean of random directions: $\mathbf{E}[\widetilde{\nabla} E(\boldsymbol{w})] = \nabla E(\boldsymbol{w})$
- …but the smaller the batch size the more variance of directions
- …consequently might need more iterations to converge

gradient descent | SGD moderate batch size | SGD small batch size



Adapted from https://www.analyticsvidhya.com/blog/2022/07/gradient-descent-and-its-types/

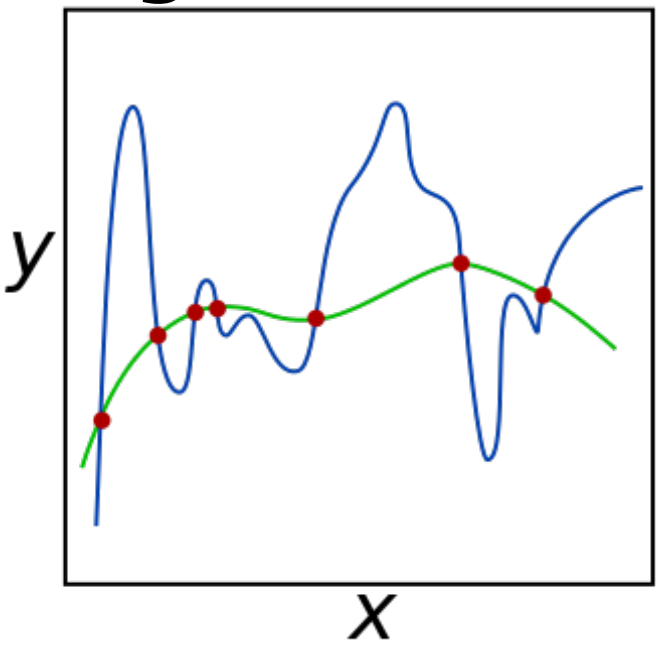
- In fact, negative stochastic gradient might not even be a descent direction
- Trade-off when choosing b



Adapted from https://pages.cs.wisc.edu/~spehlmann/cs760/_site/project/2017/05/04/intro.html

Part A

Regularisation



Parametric regression

Given

- data $D = (x_1, t_1), \dots, (x_N, t_N)$
- sampled according to joint distribution of RVs X and T
- input variable(s) X often assumed with $D_X = \mathbb{R}^d$
- real-valued target variable $T \in \mathbb{R}$

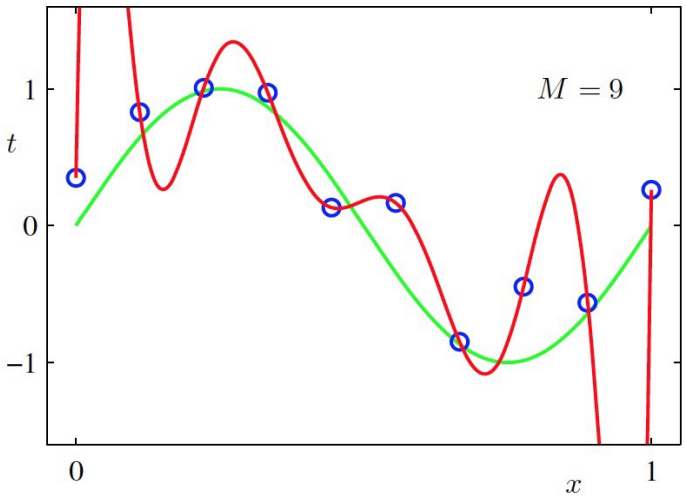
Goal / process

- learn parameters w of prediction function $y(x, w)$
- …with low generalisation error $\mathbf{E}[(y(X, w) - T)^2]$
- …via minimisation of training error $\sum_{n=1}^{N} \frac{1}{2n} \left(t_n - y(x_n, \boldsymbol{w})\right)^2$

Problem

- We know that training error is not a good estimator for generalisation error

Generalisation Performance



Problem:
training error might be weak proxy for generalisation error for complex models

$$\mathbf{E}\left[\frac{1}{2}\left(T - y(X, \boldsymbol{w})\right)^2\right] >> \sum_{n=1}^{N} \frac{1}{2n}\left(t_n - y(x_n, \boldsymbol{w})\right)^2$$

Question:
Could weights that do not minimise training error be better?

Observation

- We observed that overfitted model had very large weights
- Idea: restrict model flexibility by restricting magnitude of weights

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

Regularisation

Idea

- Minimise sum of training error and term that penalises large weights, i.e., regularised training error

$$E(\boldsymbol{w}) + \lambda\Omega(\boldsymbol{w})$$

regularisation parameter

Questions

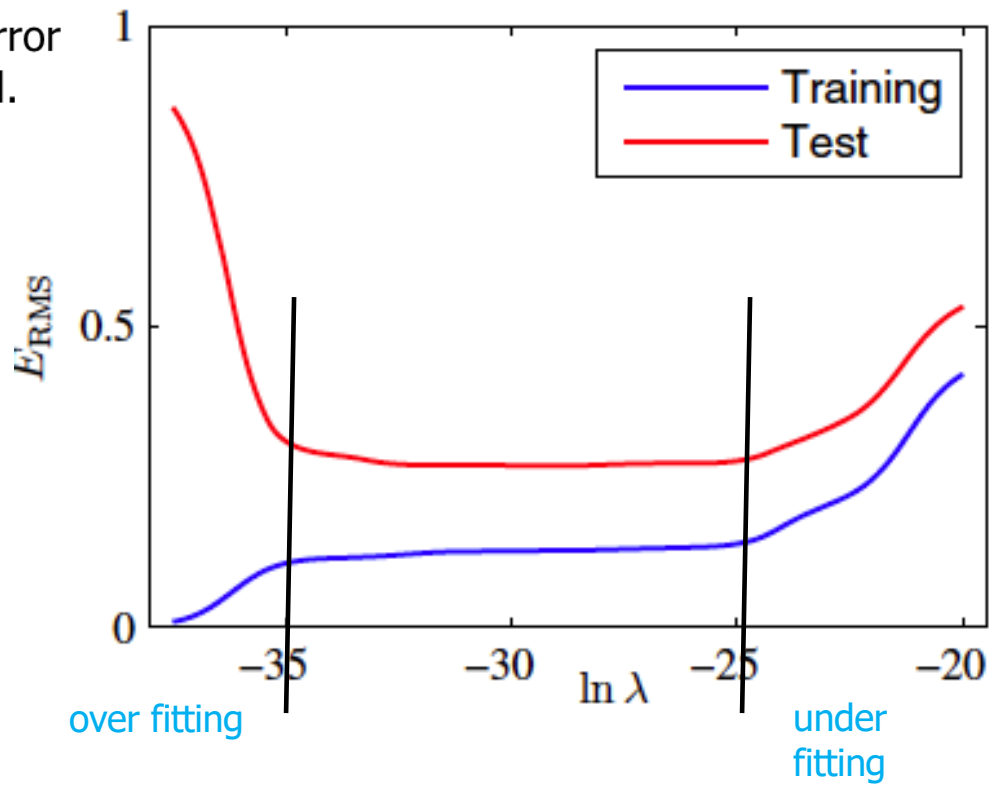
- How to choose regularisation parameter?
- How to define penalty?
- How to optimise new training objective?

How does regularisation work?

For a large λ, models with high complexity can be ruled out. For a small λ, models with high training errors can be ruled out. The optimal solution lies somewhere in the middle.

Graph of the root-mean-square error vs. $\ln(\lambda)$ for the M = 9 polynomial.

1. λ increases, model becomes less complex, training error increases
2. λ is very small, model too complex, test error high (over fitting)
3. λ increases, model becomes less complex, testing error decreases
4. λ is too large, model too simple, testing error increases (under fitting)



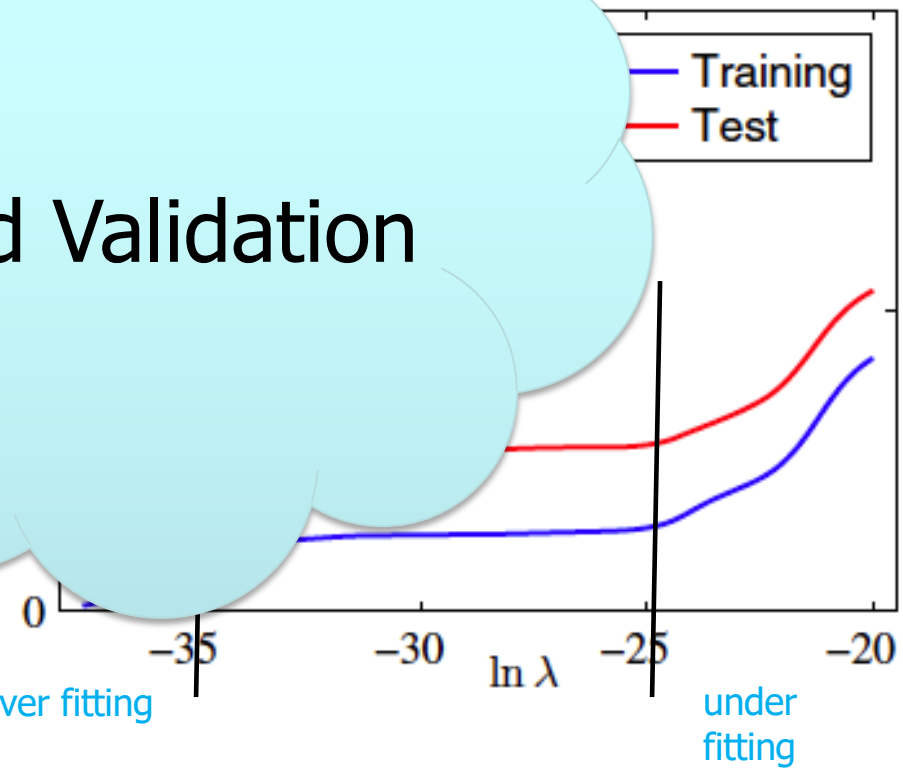
How does regularisation work?

For a large λ, models with high complexity can be ruled out. For a small λ, models with high training errors can be ruled out. The optimal solution lies somewhere in the middle.

Graph of the root-mean-s
vs. $\ln(\lambda)$ for the M = 9 p

1 λ increases, model becomes less cor training error inc
2 λ is very small, n complex, test erro (over fitting)
3 λ increases, mode becomes less complex, testing error decreases
4 λ is too large, model too simple, testing error increases (under fitting)

K-fold Validation



Penalty Function

- Typically: based on some appropriately chosen norm of weights, i.e., $\Omega(\boldsymbol{w})$ some function of a norm $\|\boldsymbol{w}\|$
- Make sure appropriately "balanced" with training error, e.g., consider sum of errors instead of mean (or scale norm by $1/n$)

Two popular examples for linear regression:

Ridge Regression

$$\frac{1}{2}\sum_{n=1}^{N}\left(t_n - y(\boldsymbol{x}_n, \boldsymbol{w})\right)^2 + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$$

Lasso Regression

$$\frac{1}{2}\sum_{n=1}^{N}\left(t_n - y(\boldsymbol{x}_n, \boldsymbol{w})\right)^2 + \frac{\lambda}{2}\|\boldsymbol{w}\|_1$$

squared Euclidian (L2) norm

$$\|\boldsymbol{w}\|_2 = \sqrt{\sum_{j=1}^{p} w_j^2}$$

Manhattan (L1) norm

$$\|\boldsymbol{w}\|_1 = \sum_{j=1}^{p} |w_j|$$

- Make sure all features have equal scale (e.g., don't measure one feature in meters and one in millimeters), i.e., normalise. Why?

How to optimise ridge objective?

For ridge regularised error is still a quadratic function of weights

$$\frac{1}{2}\sum_{n=1}^{N}\left(t_n - y(\boldsymbol{x}_n, \boldsymbol{w})\right)^2 + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$$

Finding gradient and setting to 0 again identifies system of linear equations with solution:

$$\boldsymbol{w} = (\lambda \boldsymbol{I} + \boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \boldsymbol{t}$$

Hence, can be solved directly or iteratively with (S)GD

Sidenote: Can derive ridge regression as Bayesian method by defining normal distribution prior distribution on parameters

How to optimise Lasso objective?

Problem: Lasso regularised error is not differentiable

$$\frac{1}{2}\sum_{n=1}^{N}\left(t_n - y(\boldsymbol{x}_n, \boldsymbol{w})\right)^2 + \frac{\lambda}{2}\|\boldsymbol{w}\|_1$$

However, is still convex.

Popular options:

- ❑ Use gradient descent anyway with "sub-gradient"
- ❑ Coordinate-wise descent

Key Differences: Ridge & Lasso

What are key differences between Ridge and Lasso regression

- Ridge: The main advantage is parameter shrinkage and reduce the model complexity
- Lasso: Along with shrinking parameters, it selects input variables as well. Some features can excluded from the model.

When do we have to use Ridge or Lasso?

- Ridge: Mainly used to prevent overfitting, but not useful in case of high # of input variables.
- Lasso: It provides sparse solutions, but useful in case of high # of input variables as those ones with zero parameters can simply ignored.

L1 : feature selection?

Shoe size	Weight	Height
2	1.8	4
3	3.1	6
…	…	…
4	3.9	8

h: height, the target variable
s: shoe size
w: weight

$$h = \alpha_1 s + \alpha_2 w$$

$$\Downarrow$$

$$\alpha_1 + \alpha_2 = 2$$

$$\alpha_1 = 1,\ \alpha_2 = 1 \ or\ \alpha_1 = 2, \alpha_2 = 0 \ or\ \alpha_1 = 0, \alpha_2 = 2$$

L1 : feature selection?

Shoe size	Weight	Height
2	1.8	4
3	3.1	6
…	…	…
4	3.9	8

h: height, the target variable
s: shoe size
w: weight

$$h = \alpha_1 s + \alpha_2 w$$

$$E = \sum_{i=1}^{n} (h' - \alpha_1 s - \alpha_2 w)^2 + \lambda (a_1^2 + a_2^2) \Longrightarrow \alpha_1 = 1,\ \alpha_2 = 1$$

$$E = \sum_{i=1}^{n} (h' - \alpha_1 s - \alpha_2 w)^2 + \lambda (|a_1| + |a_2|)$$

$$\Downarrow$$

$$\alpha_1 = 2, \alpha_2 = 0 \ or\ \alpha_1 = 0, \alpha_2 = 2$$

Summary/Recap

- Form of linear model with basis functions
- Squared error minimisation is maximum likelihood maximisation
- Characterisation of optimal parameters via system of linear equations ("normal equations" from derivative criterion)
- Gradient Descent (GD) algorithm and Stochastic GD (SGD) algorithm
- Regularisation (Ridge + Lasso)

Laboratory (Week 3)

Implement learning algorithms for linear regression:

- ❑ Direct approach (normal equations)
- ❑ Gradient descent
- ❑ Stochastic gradient descent

Relevant Reading

- Pattern Recognition and ML, Chap 3.1
- Elements of Statistical Learning, 3.1, 3.2

What will we learn in Week 4

Part B in Module 2

- ☐ Deeper look into overfitting phenomenon
- ☐ Decompose squared error into bias and variance
- ☐ Use bias/variance-analysis to derive modelling recommendations